



OPEN Fuzzy logic applied to tuning mutation size in evolutionary algorithms

Krzysztof Pytel

Tuning of parameters is a very important but complex issue in the Evolutionary Algorithms' design. The paper discusses the new, based on the Fuzzy Logic concept of tuning mutation size in these algorithms. Data on evolution collected in prior generations are used to tune the size of mutations. A Fuzzy Logic Part uses this historical data to improve the algorithm's convergence to a global optimum. The Fuzzy Logic Part keeps a desirable relation of exploration and exploitation, so the algorithm's resistance to getting stuck in a local optimum is improved too. Several tests on Function Optimization Problems were performed to prove the suitability of the proposed method. A set of data and functions with different difficulties, recommended in the commonly used benchmarks are used for experiments. The results of these experiments suggest that the proposed method is efficient and could be used for a wide range of similar problems of optimization.

Keywords Optimization, Evolutionary algorithm, Function optimization

There are various optimization metaheuristics inspired by observations of nature and biological/social behaviors. Particle Swarm Optimization (PSO)¹ draws inspiration from the swarm of birds or fishes' collective behaviors. Artificial Bee Colony (ABC)² is inspired by the behavior of honeybees while foraging. Social Spider Optimization (SSO)³ is a metaheuristic inspired by the communication, cooperation, coordination, and other complex social interactions and hunting behavior of spiders. In Ant Colony Optimization (ACO)⁴ ants use pheromone trails to communicate and find the best path. Grey Wolf Optimization (GWO)⁵ is inspired by the social hierarchy and hunting behavior observed in wolf packs. In Evolutionary Algorithms (EA)^{6,7} a population of candidate solutions evolves through mutation, crossover, and selection. They use concepts and terminology from biology, such as gene, population, selection, reproduction, and mutation. More information about EA and details of their construction can be found in publications^{6,7}.

EA executes processes of exploration (searching for a new region where an optimum may exist) and exploitation (searching the local neighborhood). Maintaining the right relationship between exploration and exploitation throughout EA running is very important. It can collect information about the process of evolution and use this historical information to tune its parameters, e.g. mutation size. This information is descriptive and can be expressed with IF . THEN . rules. The idea and Fuzzy Logic (FL) theory was first presented in⁸. It is a kind of multi-valued logic that uses the partial truth idea, and values ranging from 0 to 1. The Linguistic Variable (LV) expresses states or rules, e.g., a number of mutations may accept values: small, medium, or high. Each LV is associated with one or more Membership Functions (MFs), that define how input values are mapped to degrees of membership within the fuzzy set. Fuzzy logic theory and its applications are extensively covered in publications such as^{9,10}. Fuzzy Logic Part (FLP) can control parameters in EAs, for example in the following publications^{11–13}. A few other methods to improve EA effectiveness have been presented, e.g. the literature^{14,15}.

The Evolutionary Fuzzy Systems (EFSs) represent a hybridization of FL and optimization based on evolutionary metaheuristics inspired by an observation of nature^{16,17}. The EFSs approaches can be broadly categorized into two main classes: tuning and learning. Publication¹⁸ presents an example of the learning approach, where the Takagi-Sugeno Fuzzy inference system is used to capture, replicate, and adapt a human expert's search strategy. The new proposed EA that uses an FLP to tune mutation size based on historical data introduces several unique aspects that distinguish it from conventional approaches described in the literature:

- FL adds a new level of adaptability and flexibility to the algorithm. The FLP is applied to dynamically manage the mutation size of the EA, to keep a proper balance between the algorithm's ability to explore and exploit.

Faculty of Physics and Applied Informatics, University of Łódź, Pomorska 149/153, Łódź 90-236, Poland. email: krzysztof.pytel@fis.uni.lodz.pl

- Algorithm incorporates expert knowledge and historical data. The FLP uses experts' knowledge about evolution and information collected in previous generations (historical data) to adjust the mutation size in the EA.
- User-defined modification of historical data size. The size of historical data, for example, the number of generations analyzed by the FLP provides flexibility. It can be modified by the user and adapted to the specific requirements of the optimized problem.
- The proposed method is not limited to mutation size modification. It can be extended to alter other parameters, for example, probability of selection. This broad applicability increases the versatility of the approach for optimizing various aspects of the algorithm.

The field of EAs has seen the development of various methods that leverage historical data to guide the evolution process. A review of methods for guiding evolutionary algorithms is available in the publication¹⁹. Detailed information on different approaches to the adaptation of EA parameters is available in publications^{20–24}. The proposed method introduces notable distinctions from previously presented methods. These differences likely contribute to the superiority of the proposed approach. Details on these differences will be provided further on, including experimental results, comparisons with existing methods, and discussions on the advantages and limitations of the proposed approach. The main contribution of the paper is:

- A new EAs model, where the FLP tunes the mutation size is presented;
- The paper identifies and proposes estimators representing input values for the FLP. These estimators play a crucial role in the FLP's decision-making process;
- Experiments on a set of benchmarks confirm the superiority of the proposed system over traditional methods in both the convergence and resistance to getting stuck in the local optima.

The contributions of the paper in comparison to the author's previous works^{25,13}, can be summarized as:

- The paper introduces a novel method for controlling evolution in EAs by modifying the mutation size;
- The paper conducts an analysis and selection process for estimators that play a crucial role in determining the direction of evolution;
- The paper proposes the construction of a FLP, based on the selected estimators. The integration of FL into the decision-making process of the EA represents a significant enhancement that offers several advantages in terms of performance and efficiency;
- The proposed method's effectiveness was tested, and the results were compared with the algorithms presented in the previous works. These results demonstrate its advantages over earlier methods.

The article's structure is outlined across five sections. Sect. "[Introduction](#)" provides an introduction to FL and EAs and serves as a foundation for understanding the subsequent discussion on the integration of these two methodologies. It presents concepts related to guided evolution in EAs and introduces the fundamental concepts and motivations for the research. This section reviews existing methods used to optimize and tune the parameters in EAs. Sect. "[Problem formulation and new method for tuning mutation size in evolutionary algorithms](#)" justifies the choice of the Function Optimization Problem (FOP) as a benchmark for evaluating the proposed algorithm by highlighting its suitability, versatility, ease of evaluation, and relevance to real-world optimization tasks. This section provides the necessary background and context for understanding the experimental evaluation presented in later sections of the paper. It introduces a new method of parameter tuning that uses an FLP and incorporates information from the history of evolution. Sect. "[Computational experiments](#)" provides a detailed description of the computational experiments conducted to evaluate the proposed algorithm, and presents empirical evidence to support its effectiveness, efficiency, and applicability in solving optimization problems, including real-life problems. It describes the benchmark functions and new code used in the experiments. This section serves as a reference for readers interested in the technical implementation details of the proposed method. Sect. "[Application of the proposed algorithm to computer network infrastructure optimization](#)" describes the use of the proposed system to solve the ConFLP problem. In this problem, the algorithm optimizes the computer network structure and determines network node locations. Sect. "[Conclusion](#)" summarizes the overall work in the article and presents the conclusions from the experiments. It proposes future developments, indicating potential areas for further research or improvements to the proposed method.

Problem formulation and new method for tuning mutation size in evolutionary algorithms

The aim of optimization is to find the best object (or group of best objects) from a set of objects, concerning one or multiple criteria. In an FOP, the parameters (variables) of the optimized function must be fitted to obtain the best value of the objective function. The FOP problem (considering the maximization problem) is stated as follows:

$$\begin{cases} \max f(x) \\ \text{subject to: constraints } c_i \text{ for } i = 1, 2, \dots, k \\ x \in S \end{cases} \quad (1)$$

where:

$x = [x_1, x_2, \dots, x_n] \in \mathfrak{R}$, $n \in N$ - is vector of decision variables in an n -dimensional domain,
 $f(x)$ - is the objective function,
 c_i - are the constraints,

S - the search area.

The FOP is commonly used as a benchmark to evaluate various optimization methods, including EAs, metaheuristic algorithms, and other optimization techniques. A lot of methods, that solve the FOP have been proposed in the literature, for example^{2,26–28}.

A few EA variants are presented in the literature, that differ in some aspects, e.g. in the method of mutation or selection to the parent's pool. Algorithm parameters, such as the probability of selection and mutation, and population size also affect the course of optimization. During the execution of the algorithm, some of these parameters may change their values to provide better convergence or resistance to getting stuck to a suboptimal solution. Parameters can be changed according to a fixed manner or they can be subject to a tuning process. In a standard EA, decisions regarding the selection and mutation of individuals are made based on the value of their fitness function only. Information on optimization progress collected in earlier generations can be used in the tuning of algorithm parameters such as mutation size. As this knowledge of optimization progress is descriptive, rules in the style IF X AND Y THEN Z can be used to represent it and FLP can exploit this knowledge to tune the mutation size.

Gene mutation introduces a difference, in a genetic sense, between a parent and its offspring. The ability of an EA to create offspring that are genetically different from their parents depends on the size and model of the mutation. Small mutation size (makes descendants similar to their parents) is preferable in the final period of optimization when the algorithm approaches the optimum. However, the risk that the algorithm will terminate at the local suboptimum is increased. A large mutation size (makes offspring genetically distant from its parents) increases exploration, making it easier for the algorithm to locate new areas where the optimum may occur. Many EA implementations, use a reduction of the mutation size during the run of the algorithm according to a fixed function or self-adaptation rules. In the proposed algorithm, FLP tunes the mutation size, based on data collected in earlier generations. FLP calculates the new mutation size for a population in every generation, based on two estimators:

- The ratio of successful to all mutations.

$$m_r = \frac{\sum_{i=1}^k n_{sm}}{\sum_{i=1}^k n_m} \quad (2)$$

where:

m_r - the mutation ratio,
 n_{sm} - the number of successful mutations,
 n_m - the number of all mutations,
 k - the population size.

A high number of successful mutations means progress in optimization, but the mutation size could be increased to reduce computation time. A low number of successful mutations suggests that the algorithm is searching a local neighborhood. Expanding the mutation size makes it easier for the algorithm to transfer to the new region of search.

- Increment in average population fitness over history.

$$avr f = \frac{\frac{1}{h} * \frac{1}{k} * \sum_{i=1}^h \sum_{j=1}^k f(x_{i,j})}{\frac{1}{k} * \sum_{j=1}^k f(x_j)} \quad (3)$$

where:

$avr f$ - the increment of the average fitness,
 $f(x_{i,j})$ - the fitness of individual j in i generation,
 $f(x_j)$ - the fitness of j -th individual in the present generation,
 k - the population size,
 h - the history size.

The high speed of fitness growth means the progress of optimization, so the mutation size can be increased to speed up the algorithm's convergence. A low speed of fitness growth means that the algorithm is approaching the optimum and the mutation size could be decreased to improve exploitation.

The FLP tunes the mutation size (depending on data collected in previous generations) to control the balance between exploration and exploitation in an EA, through simple rules:

- Increase the mutation size, if the proportion of successful mutations to total mutations is high. This rule represents a situation where the successful to all mutations ratio is high (indicating many positive mutations). Enlarging the mutation size promotes exploration by generating more diverse individuals. With an increased search area, the algorithm can explore a broader area of possible solutions. This situation is advantageous, especially in the early stages of optimization when searching broadly is essential. However, this may reduce exploitation as the algorithm becomes less focused on fine-tuning solutions locally.
- Diminish the mutation size, if the successful to all mutations ratio is small. This rule represents a situation where the successful to all mutations ratio is small (indicating too few positive mutations). Diminishing the mutation size facilitates exploitation by producing individuals more closely related to their parents. It allows an algorithm to search the area more thoroughly, particularly in the later stages of optimization when the algorithm is close to the final solution. However, there's an increased risk of getting stuck in a suboptimal solution if the mutation size becomes too small.

- Increase mutation size if the growth of the average fitness in subsequent generations is high. This rule represents a situation where the average fitness growth speed is high (average fitness value improves rapidly over successive generations). Enlarging mutation size accelerates exploration, so an algorithm can discover new promising areas. Increased genetic diversity and the encouragement of genetically distant individuals lead to more efficient discovery of new promising regions. Especially applicable in the early period of optimization.
- Diminish the mutation size, if the fitness growth speed is low. This rule represents a situation where the average fitness growth speed is low (average fitness value changes slowly over successive generations). Diminishing the mutation size allows the algorithm to focus on fine-tuning solutions locally. Individuals operate more thoroughly within the local search space. Particularly relevant in the later stages when individuals populate the area of the optimum. However, there is a potential risk of being stuck in a sub-optimal solution.

Figure 1 presents the block diagram of the modified EA, that uses FL to tune mutation size. Estimators are calculated, based on formulas (2) and (3) and the data of the current and several last generations, depending on the history size. Tuning the mutation size starts with a generation where the increment in average fitness can be calculated. The last few generations are used for calculations, depending on the size of the history. The FLP calculates the mutation size factor applied to all individuals in the upcoming generation. The value of the factor is limited to 5% of its initial value, in order to prevent rapid changes. For example, if the ratio of successful to all mutations is high and the rate of fitness gain is high, this suggests that mutations lead to better and better solutions, so you can increase the mutation size to speed up the optimization process. If the ratio of successful to all mutations is small and the rate of fitness gain is low, this suggests that the size of mutations can be reduced so that the algorithm focuses on the local search area.

The FLP design was developed in MATLAB's Fuzzy Logic Toolbox, based on the Mamdani model and the defuzzification method "center of gravity". The input and output MFs of particular LVs are derived from statistical data obtained from initial simulations. During the initial experiments, information was collected on the individuals, such as the number of successful mutations, and the difference in average fitness between adjacent generations. This data has been used to establish input and output MFs: their number, shape, and characteristics. Due to the low computational effort and ease of implementation, a trapezoidal membership function was used. This function is represented by the formula:

$$f(x; a, b, c, d) = \max(\min(\frac{x-a}{b-a}, 1, \frac{d-x}{d-c}), 0) \quad (4)$$

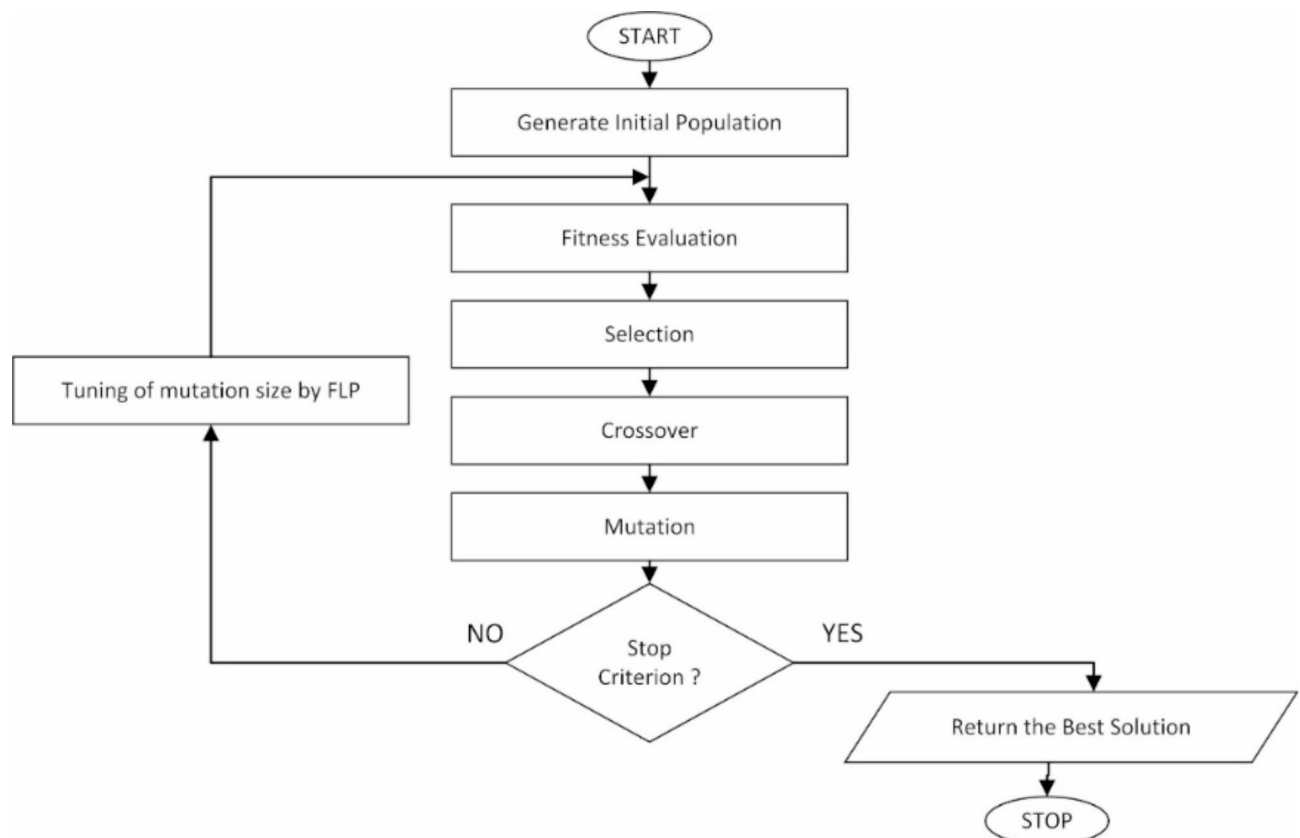


Fig. 1. The flowchart of the modified evolutionary algorithm.

Parameters a , b , c , and d were determined experimentally during initial experiments. Figure 2 illustrates examples of input and output MFs. The FLP rule database consists of a set of simple rules that reflect experts' knowledge about evolution. Examples of those rules are presented in Fig. 3. Users can select the parameters of the algorithm, such as gene representation, population size, probability of mutation, and selection, depending on the problem to be solved. Tuning of mutation size is programmable too. Users can select the history size, and MF's shape and properties. The algorithm's code was implemented in C++ language with standard libraries for Windows system.

In experiments, we use EA that searches space in the n -dimensional domain $\mathcal{R}^n$. Individuals' genes are represented as fixed-length vectors of real numbers. The fitness function $f(x)$, calculated for every individual x represents this individual's quality:

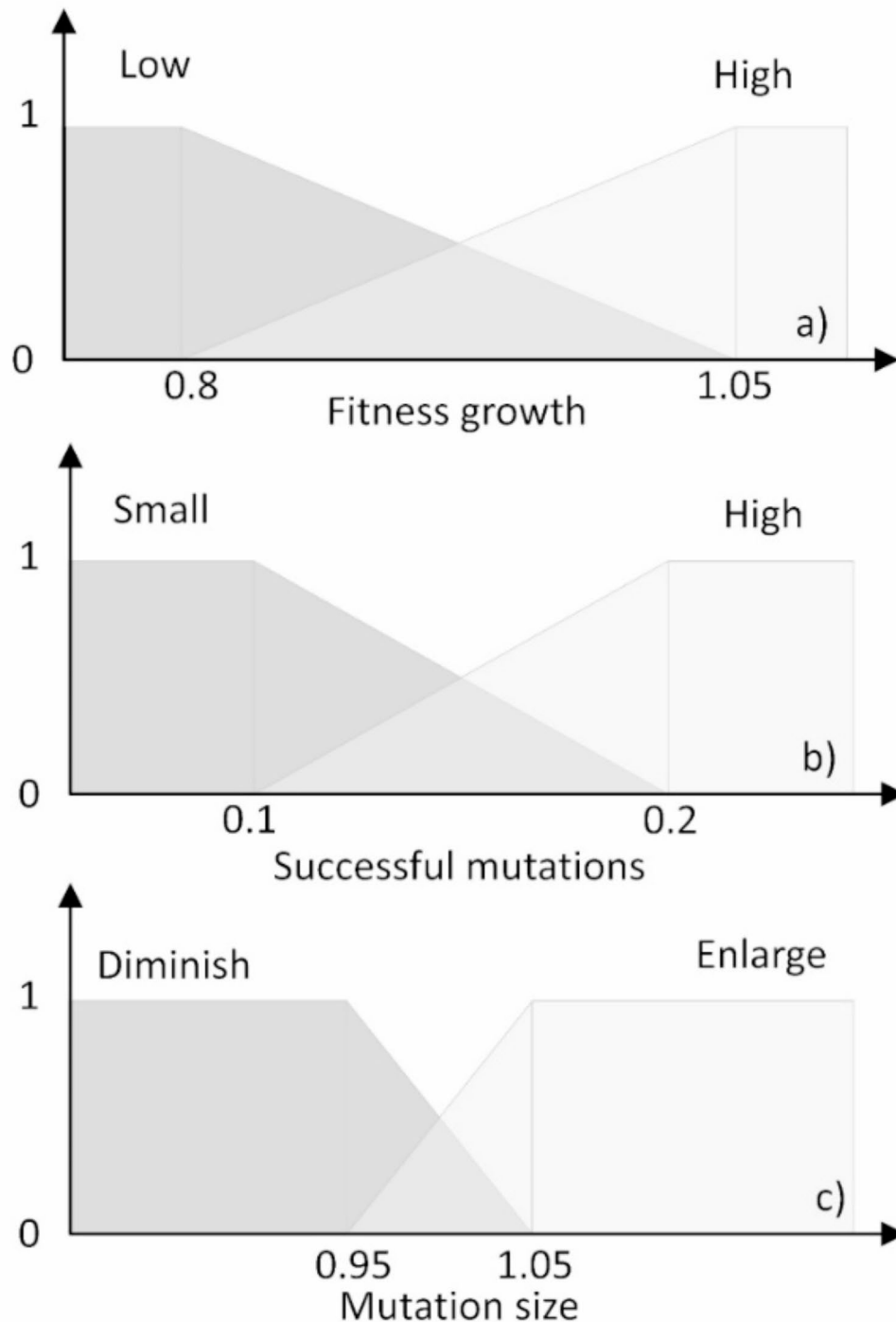


Fig. 2. The membership functions of input: fitness growth (a), successful to all mutations ratio (b), and output: mutation size (c).

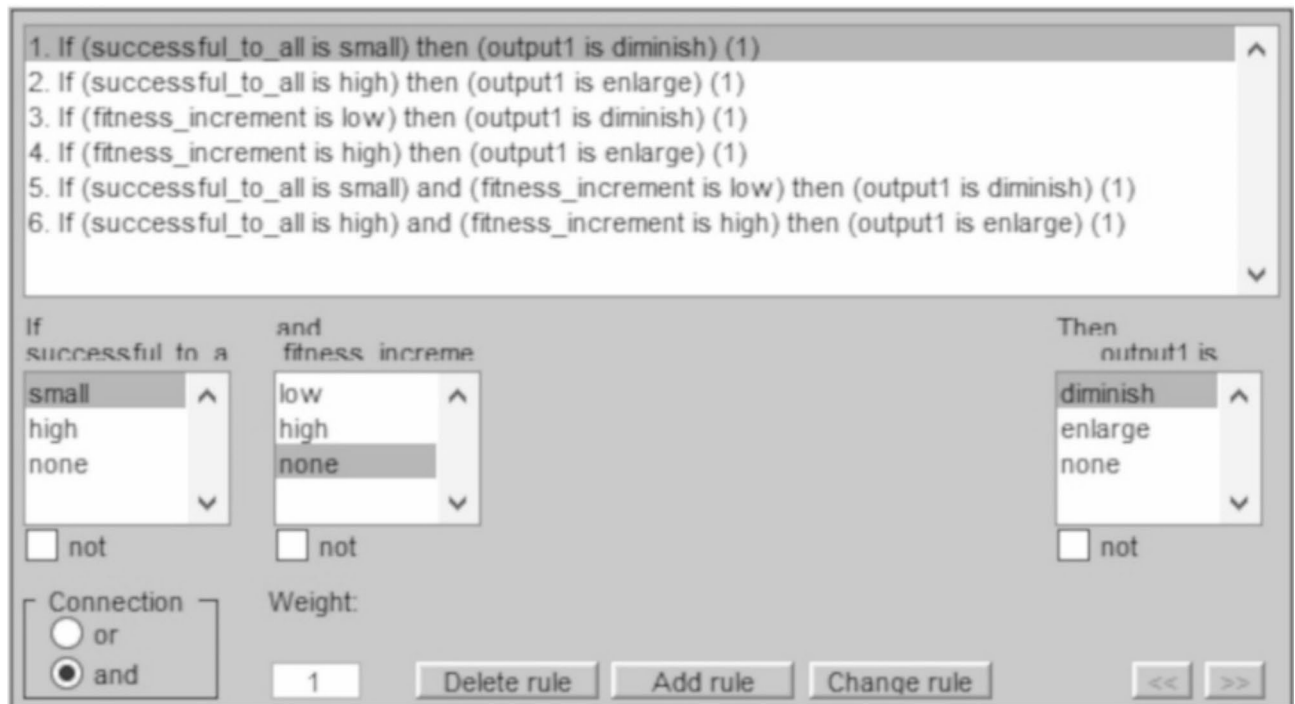


Fig. 3. The FLP rule database.

$$f(x) : \mathfrak{R}^n \rightarrow \mathfrak{R}, x = [x_1, x_2, \dots, x_n], n \in N \quad (5)$$

A mutation is made by adding a randomly generated number to the value of each individual's gene:

$$x_k^{t+1} = x_k^t + N(0, \sigma), k = 1, 2, \dots, n \quad (6)$$

The mutation process incorporates mechanisms to keep resulting gene values within acceptable domain ranges. The FLP tunes the mutation size (σ in formula 6) to improve the algorithm's performance. The algorithm's parameters, such as selection and mutation probability were established experimentally during initial experiments for each benchmark task. The algorithm terminates when the best individual reaches a predetermined value of fitness. The other algorithm's parameters used in experiments, common to all algorithms include:

- The probability of crossover = 0.8,
- The probability of mutation = 0.15,
- The population size = 25.

Computational experiments

FOP, because of its ease of implementation and well-documented set of test functions, is commonly used as a benchmark. A collection of tasks should provide neutral conditions for all compared algorithms, and contain functions with various solution difficulties, number of dimensions, number of local or global optima, etc. To test the idea of tuning mutation size in EA, the FOP problem is selected and a set of functions with a wide range of complexity:

- CEC2015 test functions²⁹ are minimization problems, with minimum shifted to 0, and search range [-100 100] for each d dimension. In experiments *CEC_f1*, *CEC_f2*, *CEC_f3* in 50 and 100 dimensions has been used:

CEC_f1($x_1, x_2, \dots, x_d$) - High Conditioned Elliptic Function, defined as:

$$CEC_f1(x_1, x_2, \dots, x_d) = \sum_{i=1}^d (10^6)^{\frac{i-1}{d-1}} x_i^2 \quad (7)$$

CEC_f2($x_1, x_2, \dots, x_d$) - Cigar Function, defined as:

$$CEC_f2(x_1, x_2, \dots, x_d) = x_1^2 + 10^6 \sum_{i=2}^d x_i^2 \quad (8)$$

CEC_f3($x_1, x_2, \dots, x_d$) - Discus Function, defined as:

$$CEC_f3(x_1, x_2, \dots, x_d) = 10^6 x_1^2 + \sum_{i=2}^d x_i^2 \quad (9)$$

- $f1(x_1, x_2, \dots, x_d)$ a product of sines. A function of 5 to 20 variables has been used to test the algorithm's ability to determine the exact solution. Its formula is:

$$f1(x_1, x_2, \dots, x_d) = \prod_{i=1}^d \sin(x_i), \quad x_1, x_2, \dots, x_d \in (0, \pi), \quad d = 5, 10, 20 \quad (10)$$

- Rastrigin function - in experiments, we used 2, 5, and 10 dimensions on the hypercube $x_i \in [-5.12, 5.12]$. The function minimum $f_{Ra}(x^*) = 0$ is at point $x^* = (0, \dots, 0)$, and its formula:

$$f_{Ra}(x) = 10d + \sum_{i=1}^d [x_i^2 - 10 \cos(2\pi x_i)] \quad (11)$$

- Styblinski-Tang function - in experiments, we used 2, 5 and 10 dimensions where $x_i \in [-5, 5]$. The function minimum $f_{St}(x^*) = -39.16599d$ is at point $x^* = (-2.903534, \dots, -2.903534)$, and its formula:

$$f_{ST}(x) = \frac{1}{2} + \sum_{i=1}^d [x_i^4 - 16x_i^2 + 5x_i] \quad (12)$$

- Rosenbrock function: in experiments, we used 2 dimensions in domain $x_i \in [-5, 10]$. The function minimum $f_{Ro}(x) = 0$ is at point $x^* = (1, \dots, 1)$ and its formula:

$$f_{Ro}(x) = \sum_{i=1}^{d-1} [100(x_{i+1} - x_i^2)^2 + (x_{i+1})^2] \quad (13)$$

- Shubert function: in experiments, we used 2 dimensions in domain $x_i \in [-2, 2]$. The function minimum $f_{Sh}(x) = -186.7309$ and its formula:

$$f_{Sh}(x) = \left(\sum_{i=1}^5 i \cos((i+1)x_1 + i) \right) \left(\sum_{i=1}^5 i \cos((i+1)x_2 + i) \right) \quad (14)$$

- Ackley function - in experiments, we used 2 and 30 dimensions, where $x_i \in [-30, 30]$. The minimum value is $f_{Ac}(x^*) = 0$ at point $x^* = (0, \dots, 0)$ and its formula:

$$f_{Ac}(x) = -20 \exp \left(-0.2 \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2} \right) - \exp \left(\frac{1}{d} \sum_{i=1}^d \cos(2\pi x_i) \right) + 20 + e \quad (15)$$

- $f2(x_1, x_2)$ simple function of two variables in domain $x_i \in [0, \pi]$. The function maximum $f2(x^*) = 1.6$ is at point $x^* = (\frac{\pi}{2}, \frac{\pi}{2})$ and its formula:

$$f2(x_1, x_2) = (\sin(x_1) + 0.6(\sin(20x_1))^* \sin(x_2) \quad (16)$$

- $f3(x_1, x_2)$ a function given by the formula:

$$f3(x_1, x_2) = \sum_{i=1}^7 h_i^* e^{-\mu_1^* ((x_1 - x_{i1})^2 + (x_2 - x_{i2})^2)} \quad (17)$$

where:

$$h_1 = 1.5, h_2 = h_3 = h_4 = 1, h_5 = h_6 = 2, h_7 = 2.5$$

$$\mu_1 = \mu_2 = \mu_3 = \mu_4 = \mu_5 = \mu_6 = \mu_7 = 0.01$$

$$(x_{11}, x_{12}) = (5, 5), (x_{21}, x_{22}) = (5, 30), (x_{31}, x_{32}) = (25, 25), (x_{41}, x_{42}) = (30, 5), (x_{51}, x_{52}) = (50, 20), (x_{61}, x_{62}) = (20, 50), (x_{71}, x_{72}) = (50, 50)$$

The value of maximum 2.5 at point (50,50).

The experiments, divided into a few stages, were done on a computer running Windows 10, equipped with an Intel i7 processor, supported by 8 GB of RAM. During initial tests, the algorithm's parameters, for example, population size, probability of selection and mutation, that best fit the test tasks were found. In the first stage of experiments, the efficiency of the algorithms to find the exact value of the optimum was investigated on monotone and easy-to-optimize function $f1$ of 5, 10, and 20 variables. The results of the proposed algorithm, that uses FLP (FLP-EA), were compared to the standard algorithm (EA)⁷. Table 1 presents the results from 30 algorithm runs: the number of generations, the algorithm running time till termination, and their standard deviations.

Algorithm	Dim. size	Stop crit.	The number of generations				The running time [s]			
			Min value	Avg value	Max value	σ	Min value	Avg value	Max value	σ
EA	10	0.999999	4693	10898.5	24,386	5708	0.267	0.5406	1.224	0.278
	20	0.999999	21,481	46591.7	72,196	14,013	1.807	3.6663	5.448	1.072
	10	0.999999	207,270	250109.0	345,307	42,042	29.057	35.7162	49.382	6.392
FLP-EA	10	0.999999	3895	8815.6	13,229	3341	0.289	0.5969	1.051	0.234
	20	0.999999	25,511	35455.9	47,134	7327	2.638	3.7779	5.003	0.812
	10	0.999999	151,457	233150.1	284,534	46,111	29.304	44.7471	54.364	8.766

Table 1. The results for optimization of $f1$: the number of generations and the running time.

Algorithm	Function	Dim. size	Stop crit.	The number of generations				The running time [s]			
				Min value	Avg value	Max value	σ	Min value	Avg value	Max value	σ
EA	CEC_ $f1$	50	1	31,510	33,922	35,905	1358	6.55	6.964	7.4	0.30
		100	10	37,877	40,511	43,039	1891	18.00	19.180	20.7	0.77
	CEC_ $f2$	50	30	863,016	1,046,476	1,210,955	97,090	144	175.1	201	16.34
		100	1000	858,736	904,124	1,056,167	1891	259	273.1	318	0.77
	CEC_ $f3$	50	1	31,464	33,340	36,243	1816	5.4	6.04	7.3	0.61
		100	1	60,587	66,627	77,322	4766	18.7	20.73	23.6	1.81
FLP-EA	CEC_ $f1$	50	1	24,028	27,827	29,046	1497	6.00	6.800	7.1	0.33
		100	10	37,637	39,195	43,114	1591	16.9	17.560	19.3	0.73
	CEC_ $f2$	50	30	786,195	946,412	1,045,048	87,932	139	167.3	184	15.23
		100	1000	679,545	775,177	918,691	1591	222	252.7	298	0.73
	CEC_ $f3$	50	1	25,541	28,735	30,387	1599	4.7	5.28	5.6	0.28
		100	1	53,661	57,808	64,345	3019	17.8	18.95	20.8	0.87

Table 2. The results for optimization of CEC functions: the number of generations and the running time.

In the second stage of experiments, a set of functions from the CEC2015 benchmark with a high number of dimensions was studied. In Table 2 the results of 30 runs of algorithms for the optimization CEC_ $f1$, CEC_ $f2$, CEC_ $f3$ functions are presented.

In the third stage of experiments, a set of difficult-to-optimize functions was studied. In Table 3 the results of 30 runs of algorithms for the optimization Rastrigin, Styblinski-Tang, Ackley, Rosenbrock, and Shubert functions are presented.

During the fourth stage, the ability of the algorithms to leave the local optimum region, on multi-modal functions $f2$ and $f3$ was investigated. The $f3$ function is challenging to optimize due to its landscape. The algorithm starts with an initial population in the local maximum at point (5,5) must pass through valleys, bypass local maxima and populate global maximum region at point (50,50). Table 4 presents the results from 30 algorithm runs on functions $f2$ and $f3$.

The median and p-value of the Wilcoxon rank-sum test are presented in Table 5. Cells with p-values below the level $\alpha = 0.05$ are marked in gray. The Wilcoxon rank sum tests prove that the results of the proposed FLP-EA compared to the standard EA algorithm are statistically significantly different. The results at the level $\alpha = 0.05$ were met in 13 out of 21 (62%) testing functions for the number of generations and in 8 out of 21 (38%) tests for the running time. Tuning the mutation size reduces the number of generations to the stopping criterion, but additional computational effort increases the running time of the algorithm. The effectiveness of adaptation depends on both the complexity of the tested function and the number of dimensions.

The optimization results for the $f3$ function, obtained by the proposed algorithm (FLP-EA), were compared with other similar methods. The Simple Genetic Algorithm (SGA), presented in publication⁷, the Simple Genetic Algorithm with the Adaptation of Probability of Selection (SGA-FLP), introduced in publication¹¹, and the Hybrid Genetic Algorithm with Evolutionary Strategy ($\mu + \lambda$)GA - ES, described in publication¹³ were selected for comparison. All parameters of the SGA, SGA-FLP, and GA-ES algorithms are the same as those presented in the source publications. Table 6 presents the number of calls to the fitness function and the running time of the compared algorithms.

The Rosenbrock function is a popular test problem for optimization algorithms. The function is continuous and unimodal. Its global minimum is in a narrow, parabolic valley, but convergence to the minimum is difficult. This function represents the unimodal optimization environment well, so it will be used to illustrate the changes in the mutation size in FLP-EA. In the proposed algorithm, the size of the mutation can be managed by FLP but it is limited and must be within a certain range. The course of the mutation size change for the Rosenbrock function in 2 and 30 dimensions is shown in Fig. 4. In the initial period, the mutation size is increased (an increased exploration) but in the further generations, the mutation size is reduced to enhance the algorithm's exploitation.

Algorithm	Function	Dim. size	Stop crit.	The number of generations				The running time [s]			
				Min value	Avg value	Max value	σ	Min value	Avg value	Max value	σ
EA	Rastrigin	2	0.001	4683	8841.4	20,323	4544	0.161	0.2887	0.542	0.121
		5	0.001	19,775	50067.1	84,228	21,334	0.915	2.3650	3.989	0.981
		10	5	62,918	134660.6	329,182	80,650	5.037	10.6129	25.644	6.230
	Styblinski-Tang	2	-78.331	977	2094.5	3711	1048	0.058	0.0851	0.122	0.022
		5	-195.82	3321	10465.7	13,956	3941	0.200	0.5609	0.732	0.196
		10	-391.65	20,455	39374.8	67,366	17,382	1.484	2.8413	4.796	1.236
	Ackley	2	0.0001	192,714	615365.8	1,222,510	312684.1	1.518	4.7845	9.401	2.4039
		30	0.01	660,242	1,249,333	1,726,347	387053.9	28.56	54.079	74.67	16.594
	Rosenbrock	2	0.0001	2626	5091.1	7652	1885	0.089	0.1384	0.195	0.037
	Shubert	2	100	4212	9056.4	16,943	4164	0.232	0.4498	0.891	0.206
FLP-EA	Rastrigin	2	0.001	664	532,,3	10,806	3035	0.134	0.2282	0.345	0.076
		5	0.001	17,458	34137.9	58,417	12,262	1.034	2.0275	3.353	0.713
		10	5	64,191	98649.5	153,110	27,637	6.672	10.3188	16.305	2.976
	Styblinski-Tang	2	-78.331	220	1409.3	3150	888	0.057	0.0717	0.099	0.012
		5	-195.82	2518	3826.7	5499	1239	0.159	0.2278	0.310	0.062
		10	-391.65	23,947	34156.7	45,344	7572	1.872	2.6587	3.552	0.617
	Ackley	2	0.0001	125,018	392683.1	670,651	216006.3	0.979	3.0457	5.163	1.6502
		30	0.01	765,797	1,101,909	1,453,766	225097.7	33.08	47.429	62.74	9.6662
	Rosenbrock	2	0.0001	1012	2706.7	5494	1651	0.056	0.0867	0.131	0.027
	Shubert	2	100	1611	5116.2	9839	2562	0.159	0.3999	0.712	0.180

Table 3. The results of optimization for functions: Rastrigin, Styblinski-Tang, Ackley, Rosenbrock, and Shubert.

Algorithm	Function	Dim. size	Stop crit.	The number of generations				The running time [s]			
				Min value	Avg value	Max value	σ	Min value	Avg value	Max value	σ
EA	f_2	2	1.596	1124	3983.4	8690	2569.03	0.072	0.1788	0.335	0.100
		2	2.5	8360	36091.7	75,261	23355.22	0.380	1.5378	3.237	0.968
FLP-EA	f_3	2	1.596	1545	2894.9	4235	860.71	0.090	0.1487	0.272	0.050
		2	2.5	4438	20143.1	47,437	13022.01	0.223	0.8755	2.002	0.532

Table 4. The results of optimization on functions f_2 and f_3 .

The Rastrigin function is a well-known and documented test problem for optimization algorithms. The function is highly multimodal and has several local minima. The locations of the minima are regularly distributed, so it well represents the multimodal optimization environment and does not favor any optimization method. Its implementation is available in many software libraries, algorithms, optimization methods, and programming languages or environments. Rastigin function in 30 dimensions was used to compare the convergence to the global optima of the proposed FLP-EA with different optimization methods. The Covariance Matrix Adaptation Evolution Strategy (CMA-ES), presented in publication³⁰, Grey Wolf Optimizer (GWO), introduced in publication³¹, and Particle Swarm Optimization (PSO) described in publication¹ were used for comparison. All CMA-ES, GWO, and PSO algorithms used default parameters for their MATLAB implementation. In Fig. 5 the algorithms' convergence is presented as a fitness value of the best individual after a fixed number of generations.

Application of the proposed algorithm to computer network infrastructure optimization

The multiple-objective optimization problem (MOP) concerns the optimization of multiple objectives that may compete or conflict with each other. Improving one objective may lead to worsening another, and finding the best trade-offs among these conflicting objectives is the essence of multi-objective optimization. In a typical MOP, the goal is to find a set of solutions, known as the Pareto optimal solutions. These solutions are not dominated by any other feasible solution regarding all objectives.

The Connected Facility Location Problem (ConFLP)³² is a practical example of a MOP. ConFLP is used in various real-world applications such as connecting computer terminals to network nodes. This problem involves determining the optimal locations for network nodes within a given geographic area. The goal is to minimize the total length of connections between these nodes and the terminals they serve while ensuring that each terminal is connected to exactly one network node. The model combines three classical problems in combinatorial optimization, which are integral to designing an efficient computer network topology:

Function	Dim. size	The number of generations			The running time [s]		
		EA Mdn	FLC_EA Mdn	p-value	EA Mdn	FLC_EA Mdn	p-value
f1	5	10046.5	9382	0.5205	0.480	0.573	0.4274
f1	10	47,749	37133.5	0.0312	3.723	4.050	0.7337
f1	20	249,438	244,648	0.9097	35.061	47.282	0.0376
CEC_f1	50	34173.5	28273.0	0.0002	6.895	6.85	0.3442
CEC_f1	100	40572.5	38908.0	0.1405	19.15	17.3	0.0011
CEC_f2	50	1046900.0	960912.5	0.0257	175.0	171.5	0.4265
CEC_f2	100	892483.0	765045.0	0.0022	270.0	249.0	0.0090
CEC_f3	50	32912.5	29332.5	0.0002	5.9	5.35	0.0027
CEC_f3	100	65121.0	57074.0	0.0004	19.85	18.75	0.0090
Rastrigin	2	8096.5	5215.5	0.0439	0.257	0.2105	0.3847
Rastrigin	5	43241.5	34879.5	0.0757	2.055	2.1055	0.6232
Rastrigin	10	119,813	95,609	0.4727	9.498	10.0030	0.5708
Styblinski-Tang	2	2111,5	1245	0.3075	0.086	0.0675	0.3443
Styblinski-Tang	5	12,285	3295	0.0028	0.651	0.2045	0.0028
Styblinski-Tang	10	32992.5	33880.5	0.9097	2.393	2.6085	0.9698
Ackley	2	578870,5	358,921	0.0376	4.463	2.788	0.2730
Ackley	30	1,336,245	1,091,258	0.0452	57.690	46.972	0.3075
Rosenbrock	2	5049	2300	0.0140	0.137	0.081	0.0126
Shubert	2	7865.5	4508	0.0312	0.383	0.348	0.5966
f2	2	3297.5	2894.5	0.6776	0.156	0.140	0.7913
f3	5	30659.5	20026.5	0.0437	1.3	0.861	0.0457

Table 5. The results of the wilcoxon rank-sum test.

Algorithm	Stop crit.	The number of call to fitness function		The running time [s]	
		Avg value	σ	Avg value	σ
FLP-EA	2.5	503,575	325,550	1.53	0.97
SGA	2.5	515,375	290,450	1,50	1,08
SGA-FLP	2.5	429,375	188,225	7,60	3,65
$(\mu + \lambda)$ GA-ES	2.5	565,194	361,504	11,72	6,72

Table 6. The results of optimization for f3 function.

- Network Nodes Location Problem: the placement of network nodes within the geographic area to minimize the lengths of links between nodes and terminals. Placing network nodes closer to terminals reduces the lengths of communication links, which in turn reduces latency and improves network performance.
- Network Nodes Number Problem: minimize the number of network nodes while ensuring effective coverage and connectivity. A smaller number of network nodes reduce infrastructure costs and simplify network management. However, reducing the number of nodes may lead to longer distances between terminals and nodes, increasing communication latency and network congestion.
- Terminal Assignment Problem: assign each terminal to the nearest network node to minimize the total length of links between nodes and terminals. However, ensuring that each terminal is assigned to the optimal node involves solving a nearest-neighbor assignment problem efficiently.

The model aims to strike a balance between minimizing infrastructure costs, reducing communication latency, and maximizing network efficiency. The formal definition of connecting computer terminals to network nodes within a given geographic area can be stated as follows:

$$\begin{cases} \min f_1(x_1, y_1, x_2, y_2, \dots, x_n, y_n) \\ \min f_2(n, x_1, y_1, x_2, y_2, \dots, x_n, y_n) \\ \text{subject to : cost and length constraints} \end{cases} \quad (18)$$

where:

- f_1 - the total link length between nodes and terminals,
- f_2 - function that assigns a terminal to a network node,
- $(x_1, y_1), \dots, (x_n, y_n) \in A$ - the position of the n node,
- $1 \leq n \leq n_{\max}$ - the number of network nodes.

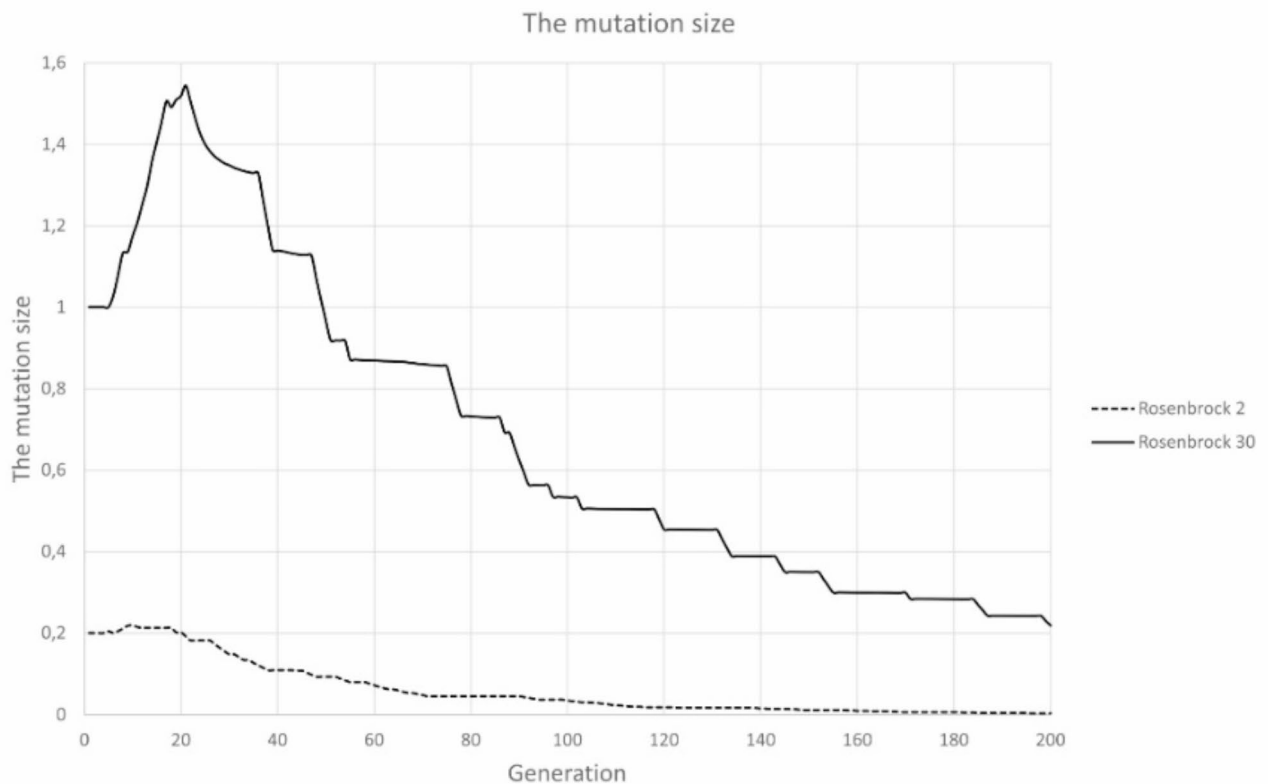


Fig. 4. The mutation size for Rosenbrock function.

In a given area of service, number of nodes, and location of the terminals, the algorithm should find the network node locations (x, y) , and associations between every terminal and only one network node. The number of terminals joined to the node is not limited. The hierarchical model in local computer networks, as proposed by Cisco, emphasizes the limitation of the node number between network extreme points. Similarly, the Spanning Tree Protocol (STP) used in layer 2 network switches is optimized for a maximum of 7 nodes between terminals to prevent loops and ensure proper network operation. With the limitations imposed by the hierarchical model and STP, traditional approaches to solving the ConFLP may struggle to find feasible solutions. The EA for solving ConFLP is proposed to find optimal solutions, that adhere to the network's structure and operational requirements. The proposed EA can determine the optimal distribution of network nodes in the given area, and assign terminals to only one, the nearest node. The algorithm encodes potential solutions using two tables:

- Table describing the position of nodes by geographical coordinates: this table contains the coordinates (x, y) of each network node, represented by real numbers. The number of genes in this table is equal to the number of network nodes.
- Table describing the connection between terminals and network nodes: this table encodes connections between terminals and network nodes using integer numbers. For example, the number i in position k means the connection between terminal k and node i . The number of genes in this table is equal to the number of terminals.

Because the ConFLP is the minimization problem, an additional constant C to convert the decreasing fitness function into the increasing function is introduced. The value of C and the value of the stopping criterion is chosen experimentally for each task. Different probabilities of mutation, are adjusted for each table of the genotype. The goal of the experiment is the verification that an EA supported by a FLP is effective in solving ConFLP. For the tests, a set of tasks from 125 to 2000 terminals were selected. The location of the terminals was randomly generated in an area of 100 by 100. The terminals are grouped into 5 clusters of equal size. The distribution of terminals and nodes after optimization for 125 terminals is presented in Fig. 6.

Each algorithm was executed 30 times. In Table 7 algorithm's number of generations to the stopping criterion and running time and presented. Graphs in Fig. 7 present the average running time and number of generations to the stopping criterion.

For the task with 500 terminals, the number of generations until the stopping criterion for the SGA algorithm increased sharply. The cause might be setting the stopping criterion close to the optimal value. In some runs, the SGA algorithm could have gotten stuck in a local optimum, and with a fixed mutation size, it had difficulty leaving it. The high standard deviation confirms the differentiation in the number of generations until the stopping criterion in individual runs. The FLP-EA algorithm, thanks to its mutation size adjustment, had a

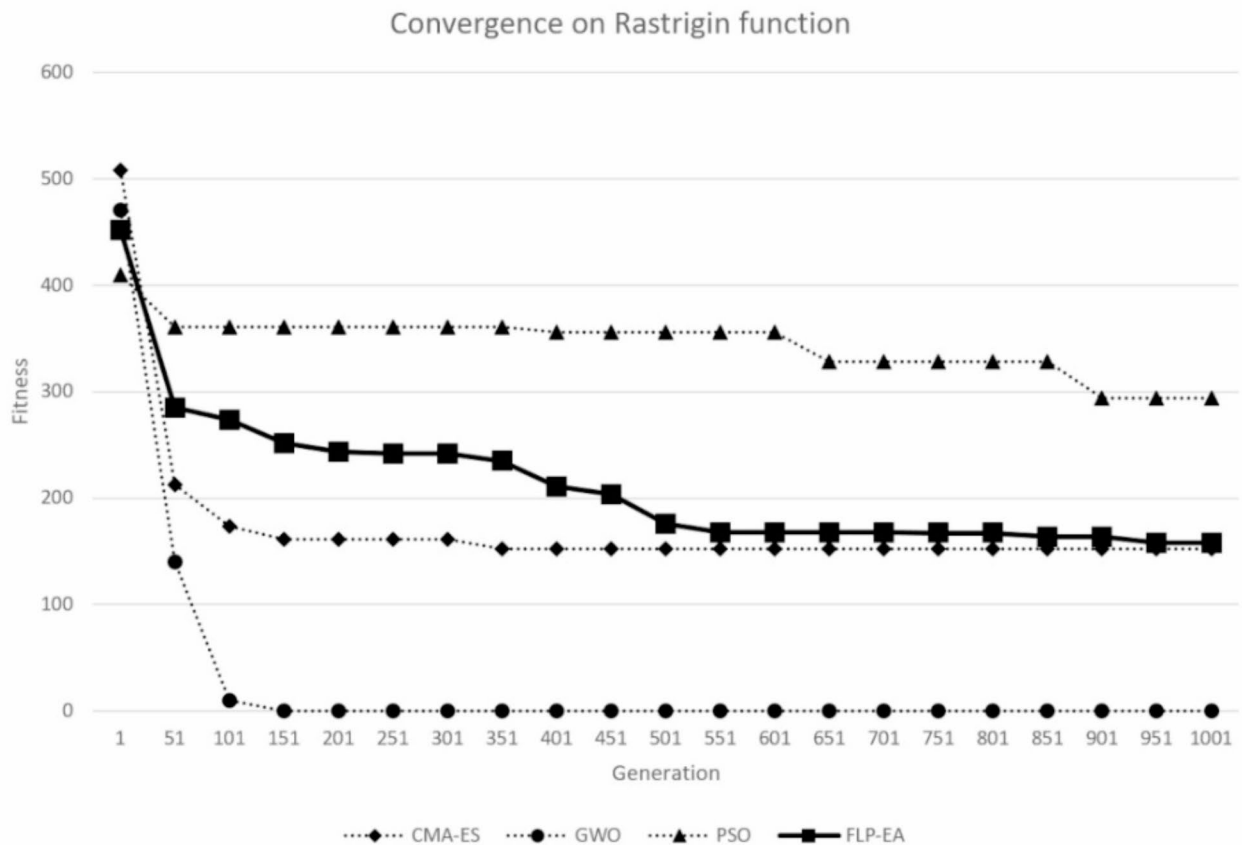


Fig. 5. The convergence of algorithms for Rastrigin function.

better chance of creating an individual capable of leaving the local optimum area, which allowed it to complete the task with fewer generations.

The FLP-EA algorithm consistently exhibits shorter running times compared to the SGA algorithm across all test task sizes. This suggests that the FLP-EA algorithm is more efficient computationally, regardless of the problem size. The reduction in running time achieved by the FLP-EA algorithm is comparable across all test task sizes. This indicates that the efficiency gain provided by the FLP-EA algorithm is consistent and not heavily dependent on the problem size. The FLP-EA algorithm requires fewer generations to reach the stopping criterion compared to the SGA algorithm. This suggests that the FLP-EA algorithm converges to a solution faster, potentially due to its more efficient exploration-exploitation strategy or its ability to maintain diversity in the population. The FLP-EA algorithm's ability to adapt the mutation probability requires additional computational effort compared to the SGA algorithm. While this adaptation may increase computational overhead, it likely contributes to the algorithm's ability to efficiently explore the search space and converge to optimal or near-optimal solutions more quickly.

Conclusion

Fuzzy Logic Part can use data about evolution collected in previous generations to tune evolution size in EA. It can effectively control the desired balance between the algorithm's ability to explore and exploit the search area. Experiments on FOP confirm that a new FLP-EA algorithm is able to solve all tasks in the benchmark.

Results from the first stage of experiments on easy to optimize function show that FLP-EA algorithm needs from 7 to 31% fewer generations than the standard algorithm. It needs more running time but improvement may occur for computationally complex functions in multidimensional environments.

Tests on multidimensional CEC2015 functions show the advantage of the proposed algorithm over the standard one, both in terms of the number of generations (from 3 to 18% better) and the running time (from 2 to 12% better).

Tests on computationally complex functions at the third stage of experiments show that FLP-EA algorithm is effective for complex functions of different dimensions. The proposed algorithm completes running after about: 26–93%, 13–63%, 36–11% 46%, 43% fewer number of generations, and after about: 2–20%, 6–59%, 36–12%, 37%, 11% shorter running time for the Rastrigin, Styblinski-Tang, Ackley, Rosenbrock, and Shubert functions, compared to the standard algorithm, respectively.

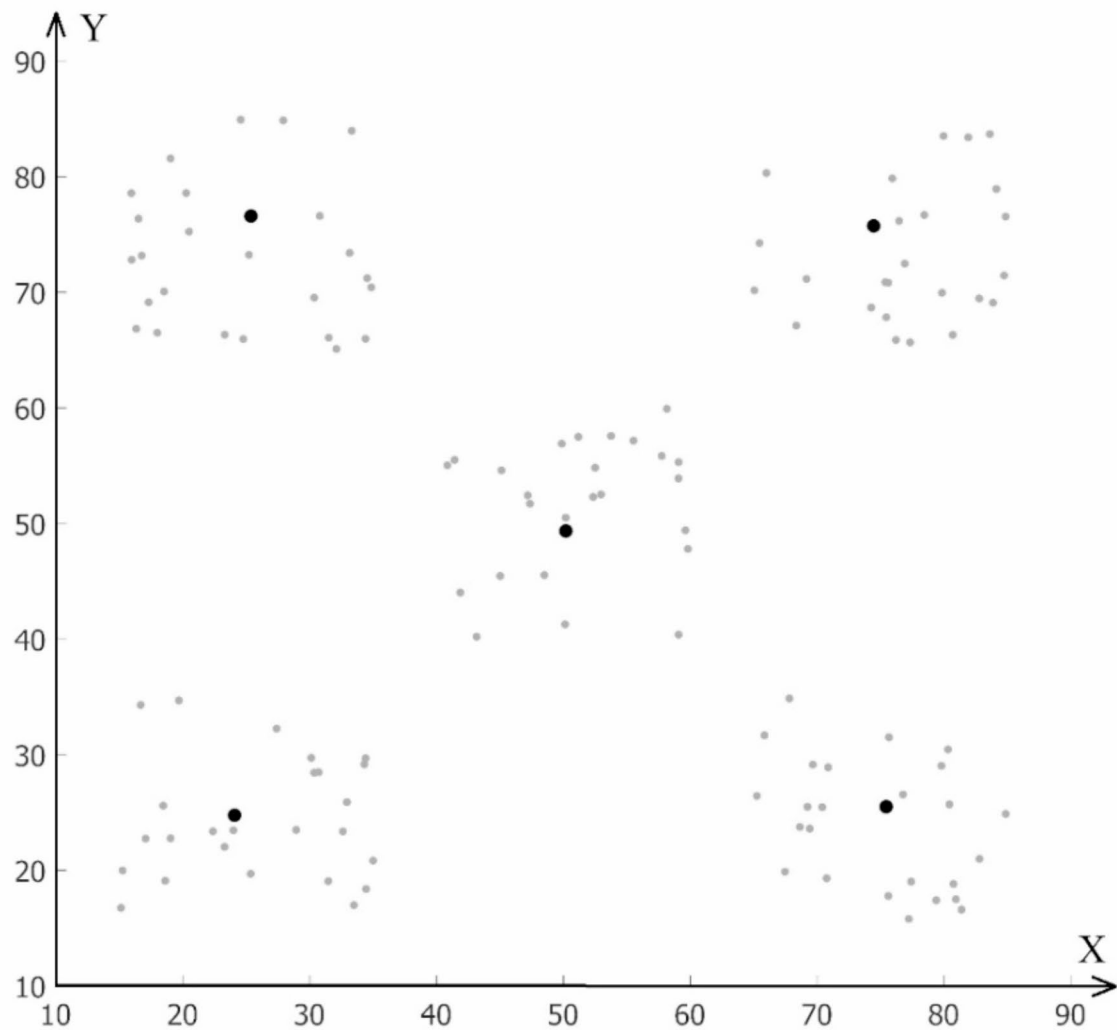


Fig. 6. Distribution of terminals and nodes after optimization.

Algorithm	Problem size	Constant C	Stop crit.	The number of generations				The running time [s]			
				Min	Avg	Max	σ	Min	Avg	Max	σ
SGA	125	7500	6300	48,010	167594.6	285,740	5.35E + 10	11	40.2	69	2941.0
	250	15,000	13,150	92,350	205769.3	483,001	1.74E + 11	40	89.1	207	33041.6
	500	30,000	25,000	213,906	433440.2	1,092,699	6.51E + 11	224	573.1	2069	2981112.0
	1000	55,000	47,000	152,301	243607.7	641,331	1.86E + 11	253	407.4	1066	508974.2
	2000	120,000	104,000	357,051	416026.1	457,087	7.98E + 09	1196	1398.5	1526	82119.5
FLP-EA	125	7500	6300	39,325	155039.1	353,226	1.10E + 11	9	36.5	82	6035.0
	250	15,000	13,150	43,102	194790.1	539,473	2.16E + 11	18	84.3	234	40804.6
	500	30,000	25,000	39,672	92292.4	243,328	2.99E + 10	34	79.2	212	23139.0
	1000	55,000	47,000	119,390	174203.2	202,177	8.71E + 09	198	290.4	335	24201.2
	2000	120,000	104,000	351,140	387902.7	461,137	1.25E + 10	1163	1319.7	1576	149230.1

Table 7. Algorithm's running time [s] and number of generations till stop criterion.

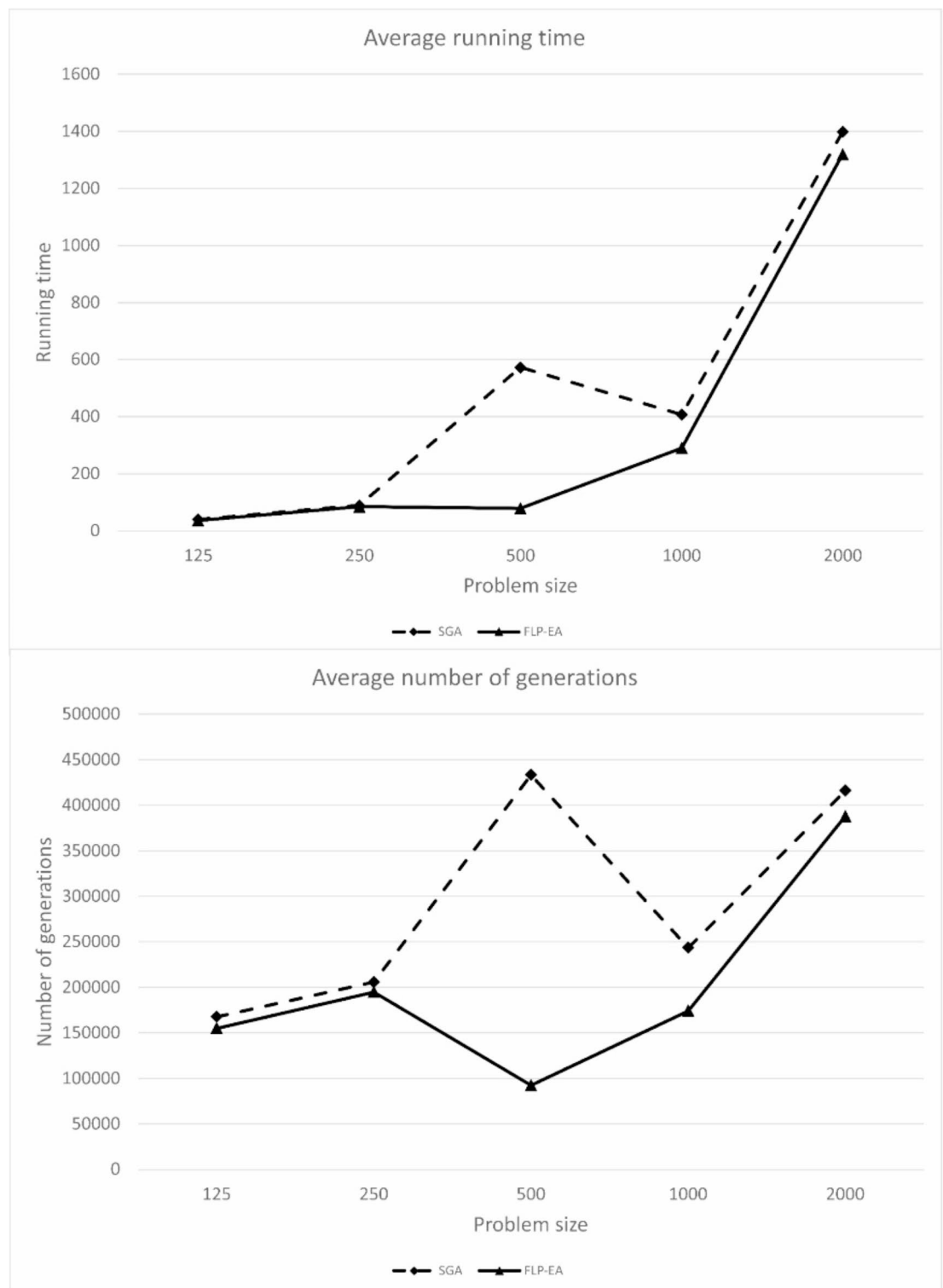


Fig. 7. Average running time and number of generations to the stop criterion.

The fourth stage of experiments shows improved exploration FLP-EA algorithm, over the standard algorithm. The new algorithm, completes tasks after 27% and 44% fewer generations and in about 16% and 43% shorter running times, for f_2 and f_3 functions, respectively.

The Wilcoxon rank sum tests prove that the proposed FLP-EA compared to the standard EA algorithm are statistically significantly different.

Based on convergence comparison, the convergence of FLP-EA algorithm is similar to CMA ES and somewhere between the GWO and PSO convergences.

The hybrid Fuzzy Logic - Evolutionary Algorithms system can offer several general advantages in solving complex problems of optimization. (A) Transparency: FL offers a transparent and easy-to-interpret way to model and express human strategies in the evolutionary process. Fuzzy rules are simple IF-THEN statements, making them easily understandable for both experts and non-experts. This transparency enhances accessibility and facilitates communication among stakeholders, promoting understanding of decision-making processes. (B) Incorporating Human Expertise: FL allows for the integration of domain-specific expertise into the optimization process. Experts can encode their domain-specific knowledge into the fuzzy rule base, to guide the EA. This ensures consistency with human goals and constraints, enhancing the relevance of the optimization process. (C) Robustness: FL enhances the system's capability to handle a wide range of conditions and uncertainties in real-world problems. By encoding expert knowledge and using linguistic variables, the system becomes more adaptable to changing scenarios and incomplete data. (D) Hybridization Benefits: The combination of FL with EAs takes advantage of the strengths of both approaches. FL provides expert knowledge-driven process control, while EAs offer search and optimization capabilities. This hybridization can significantly enhance various aspects of algorithm performance, including stability, convergence, and solution quality. (E) Adaptability: FL helps the system adapt to changing problem's conditions or environment by adjusting the fuzzy rule base or membership functions. (F) Human-Machine Collaboration: Transparency and interpretability foster collaboration between humans and machines, particularly in complex decision-making processes. Experts can work alongside the optimization system, fine-tuning rules and strategies as needed for more effective outcomes. It's emphasized that despite these advantages, designing and implementing hybrid Fuzzy Logic - Evolutionary Algorithms systems can be complex. Expertise in both FL and optimization techniques is essential, and careful consideration of the specific problem domain is crucial for the system's effectiveness.

In future research, the FLP-EA algorithm will be tested in various real-world applications such as classification problems. It is also planned to analyze the impact of various parameters of the algorithm, e.g. the size of history on the effectiveness of the algorithm, and developing hybrid algorithms where FL uses both historical data and data from the current generation to modify the probability of selection and mutation of individuals. Finally, if the tests are successful, the algorithm will be used to solve different problems of classification, for example, recognition of cosmic ray particles^{33,34}.

Data availability

The datasets generated and/or analysed during the current study are available in the Git repository, link <https://github.com/pytelek/FLP-EA>. The datasets used and/or analysed during the current study available from the corresponding author on reasonable request.

Received: 24 May 2024; Accepted: 9 January 2025

Published online: 14 January 2025

References

- Kennedy, J. & Eberhart, R. Particle swarm optimization. In: *Proceedings of IEEE International Conference on Neural Networks*. IV, 1942–1948. <https://doi.org/10.1109/ICNN.1995.488968> (1995).
- Karaboga, D. & Basturk, B. A powerful and efficient algorithm for numerical function optimization: artificial bee colony (abc) algorithm. *J. Global Optim.* **39**, 459–471 (2007).
- Cuevas, E., Cienfuegos, M., Zaldivar, D. & Perez-Cisneros, M. A swarm optimization algorithm inspired in the behavior of the social-spider. *Expert Syst. Appl.* **40** (16), 6374–6384. <https://doi.org/10.1016/j.eswa.2013.05> (2013).
- Dorigo, M., Maniezzo, V. & Colormi, A. Ant system: optimization by a colony of cooperating agents. *IEEE Trans. Syst. Man. Cybern. Part. B* **26** (1), 29–41. <https://doi.org/10.1109/3477.484436> (1996).
- Grey Wolf Optimizer (GWO). <https://www.mathworks.com/matlabcentral/fileexchange/44974-grey-wolf-optimizer-gwo>. [Accessed: 13 June 2022].
- Goldberg, D. *Genetic Algorithms in Search, Optimization, and Machine Learning* (Addison-Wesley, 1989).
- Michalewicz, Z. *Genetic Algorithms + Data Structures = Evolution Programs* (Springer, 1992).
- Zadeh, L. Fuzzy sets. *Inf. Control* **8**, 338–353. [https://doi.org/10.1016/S0019-9958\(65\)90241-X](https://doi.org/10.1016/S0019-9958(65)90241-X) (1965).
- Cox, E. *The Fuzzy Systems Handbook: a Practitioner's Guide to Building, Using, Maintaining Fuzzy Systems* (AP Professional, 1994).
- Pedrycz, W. & Gomide, F. *Fuzzy Systems Engineering: Toward Human-Centered Computing* (Wiley-Interscience, 2007).
- Pytel, K. & Nawarycz, T. Analysis of distribution of individuals in modified genetic algorithms. In: *Artificial Intelligence and Soft Computing*. (Springer, Berlin Heidelberg, 2010).
- Pytel, K. The fuzzy genetic system for multiobjective optimization. In: *Proceedings of the Federated Conference on Computer Science and Information Systems*. (IEEE Computer Society Press, IEEE Explore Digital Library, 2012).
- Pytel, K. Fuzzy-genetic approach to solving clustering problem. In: *Proceedings of the 23rd International Conference on Methods and Models in Automation and Robotics*. (IEEE Computer Society Press, Miedzyzdroje, Poland, 2018).
- Conti, E. et al. Improving exploration in evolution strategies for deep reinforcement learning via a population of novelty-seeking agents. In: *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018*. (NeurIPS 2018, Montreal, Canada, 2018).
- Wierstra, D., Schaul, T., Glasmachers, T. & Sun, T. Natural evolution strategies. *J. Mach. Learn. Res.* **15** (1), 949–980 (2014).
- Herrera, F. Genetic fuzzy systems: taxonomy, current research trends and prospects. *Evol. Intell.* **1** (1), 27–46. <https://doi.org/10.1007/s12065-007-0001-5> (2008).
- Fernandez, A., Lopez, V., del Jesus, M. J. & Herrera, F. Revisiting evolutionary fuzzy systems: taxonomy, applications, new trends and challenges. *Knowl. Based Syst.* **80**, 109–121. <https://doi.org/10.1016/j.knsys.2015.01.013> (2015).
- Diaz-Cortes, M. A., Cuevas, E., Galvez, J. & Camarena, O. A new metaheuristic optimization methodology based on fuzzy logic. *Appl. Soft Comput.* **61**, 549–569. <https://doi.org/10.1016/j.asoc.2017.08.038> (2017).
- Valdez, F., Melin, P. & Castillo, O. A survey on nature-inspired optimization algorithms with fuzzy logic for dynamic parameter adaptation. *Expert Syst. Appl.* **41**, 6459–6466. <https://doi.org/10.1016/j.eswa.2014.04.015> (2014).
- Valdez, F., Melin, P. & Castillo, O. Fuzzy control of parameters to dynamically adapt the pso and ga algorithms. In: *International Conference on Fuzzy Systems*. 1–8 <https://doi.org/10.1109/FUZZY.2010.5583934> (2010).

21. Madera, Q., Castillo, O., Garcia, M. & Mancilla, A. Interactive evolutionary computation with adaptive mutation for increasing the effectiveness of advertisement texts. In: *2016 IEEE Symposium Series on Computational Intelligence (SSCI)*. 1–6 <https://doi.org/10.1109/SSCI.2016.785000> (2016).
22. Peraza, C., Valdez, F. & Castillo, O. Interval type-2 fuzzy logic for dynamic parameter adaptation in the harmony search algorithm. In: *IEEE 8th International Conference on Intelligent Systems (IS)*. 106–112 <https://doi.org/10.1109/IS.2016.7737407> (2016).
23. Farahbakhsh, A., Tavakoli, S. & Seifolhosseini, A. Enhancement of genetic algorithm and ant colony optimization techniques using fuzzy systems. In: *2009 IEEE International Advance Computing Conference*. 336–339 <https://doi.org/10.1109/IADCC.2009.4809032> (2009).
24. Brito, F. H., Teixeira, A. N., Teixeira, O. N. & Oliveira, R. C. L. A fuzzy intelligent controller for genetic algorithms' parameters. In: (eds Jiao, L., Wang, L., Gao, X., Liu, J. & Wu, F.) *Advances in Natural Computation*, 633–642. (Springer, Berlin, Heidelberg, 2006).
25. Pytel, K. Hybrid fuzzy-genetic algorithm applied to clustering problem. In: *Proceedings of the Federated Conference on Computer Science and Information Systems, FedCSIS 2016*. FedCSIS 2016, Gdansk, Poland (2016).
26. Pytel, K. Hybrid multi-evolutionary algorithm to solve optimization problems. *Appl. Artif. Intell.* **34** (7). <https://doi.org/10.1080/08839514.2020.1730631> (2020).
27. Jensi, R. & Wiselin, G. An improved krill herd algorithm with global exploration capability for solving numerical function optimization problems and its application to data clustering. *Appl. Soft Comput.* **46**, 230–245. <https://doi.org/10.1016/j.asoc.2016.04.026> (2016).
28. Potter, M. & De Jong, K. A cooperative coevolutionary approach to function optimization. *Lect. Notes Comput. Sci.* **866** (1994).
29. Liang, J., Qu, B., Suganthan, P. & Chen, Q. Problem definitions and evaluation criteria for the cec 2015 competition on learning-based real-parameter single objective optimization. *Tech. Report201411A Comput. Intell. Lab. Zhengzhou Univ. Zhengzhou China Tech. Rep. Nanyang Technol. Univ. Singap.* **29**, 625–640 (2014).
30. Yarpiz / Mostapha Heris. Cma-es in Matlab. <https://www.mathworks.com/matlabcentral/fileexchange/52898-cma-es-in-matlab> [Accessed 16 June 2022].
31. Mirjalili, S. Grey Wolf Optimizer (gwo). <https://www.mathworks.com/matlabcentral/fileexchange/44974-grey-wolf-optimizer-gwo> [Accessed 13 June 2022].
32. Karger, D. & Minkoff, M. Building steiner trees with incomplete global knowledge. 613–623 <https://doi.org/10.1109/SFCS.2000.892329> (2000).
33. Szadkowski, Z. & Pytel, K. Trigger based on fuzzy logic for the detection of neutrino-induced extensive air showers in water-cherenkov detectors. *Eur. Phys. J. Plus* **138** (9), 809. <https://doi.org/10.1140/epjp/s13360-023-04411-5> (2023).
34. Pytel, K. Fuzzy logic applied to mutation size in evolutionary strategies. *Evol. Intel.* <https://doi.org/10.1007/s12065-023-00894-4> (2023).

Author contributions

KP all contributions.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to K.P.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025